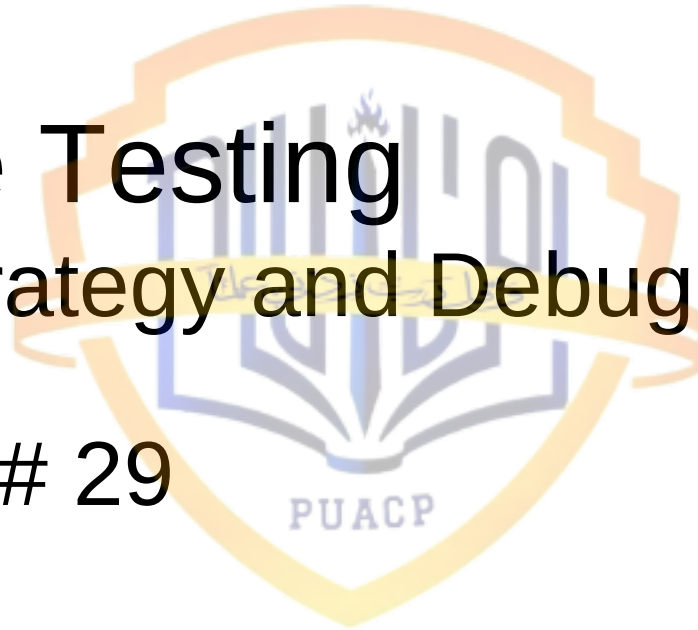




Software Testing

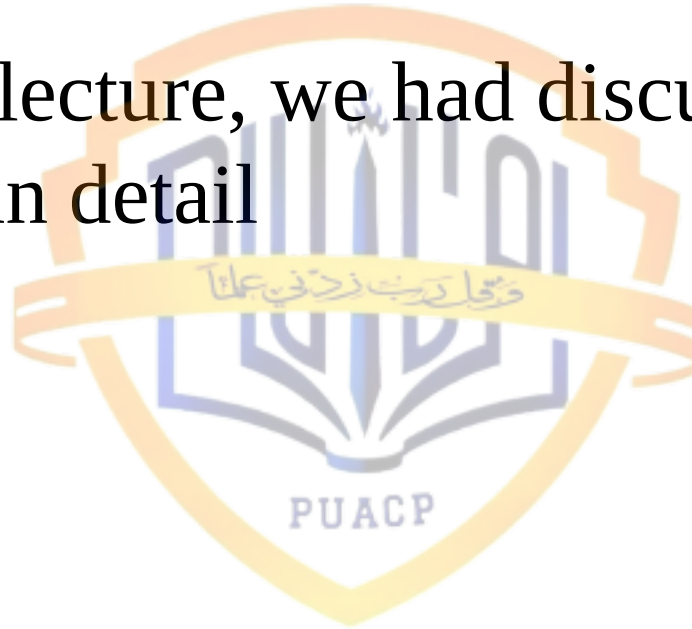
Testing Strategy and Debugging

Lecture # 29



Recap

- In the last lecture, we had discussed testing strategies in detail



Today's Lecture

- We'll talk about validation testing and debugging



Validation Testing



Validation Testing - 1

- Validation succeeds when software functions in a manner that can be reasonably expected by the customer
- Software validation is achieved through a series of black-box tests that demonstrate conformity with requirements
- A test plan outlines the classes of tests to be conducted and a test procedure defines specific test cases that will be used to demonstrate conformity to requirements

Validation Testing - 2

- Both test plan and procedure are designed to ensure that
 - All requirements are satisfied
 - All behavioral characteristics are achieved
 - All performance requirements are attained
 - Documentation is correct
 - Human engineered and other requirements are met

Validation Testing - 3

- After each validation test case has been conducted, one of two possible conditions exist
 - (1) The function or performance characteristics conform to specification and are accepted
 - (2) A deviation from specification is uncovered and a deficiency list is created

Validation Testing - 4

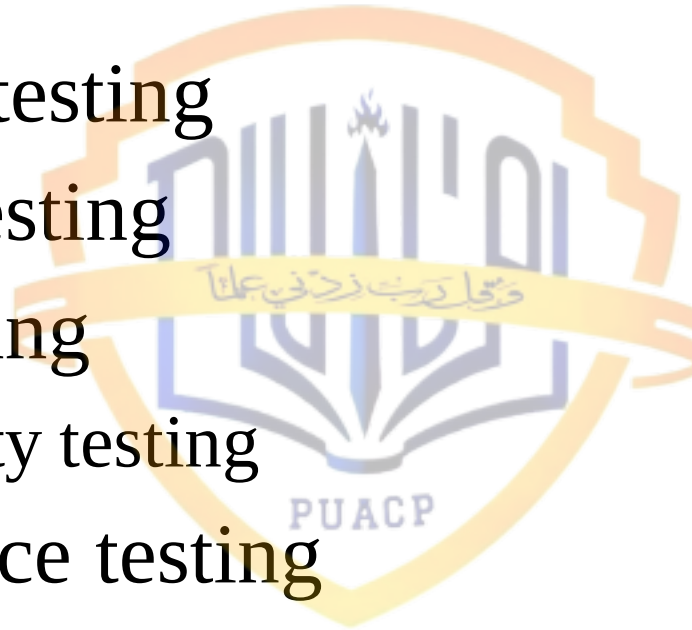
- Deviation or error discovered at this stage in a project can rarely be corrected prior to scheduled delivery
- It is often necessary to negotiate with customer to establish a method for resolving deficiencies

Other High-Order Testing

- Acceptance testing (bespoke software)
- Alpha testing (at developer's site by customer)
- Beta testing (at customer site without developer's presence)
- System testing

Specialized Testing

- Recovery testing
- Security testing
- Stress testing
 - Sensitivity testing
- Performance testing



The logo of the University of Al-Qadisiyah is a circular emblem. It features a stylized open book at the base, with a flame or torch rising from the center. Above the book, there is a banner with Arabic calligraphy. The entire emblem is enclosed within a circular border with a gear-like or architectural top. The colors are primarily blue, orange, and yellow.

Following slide to be inserted

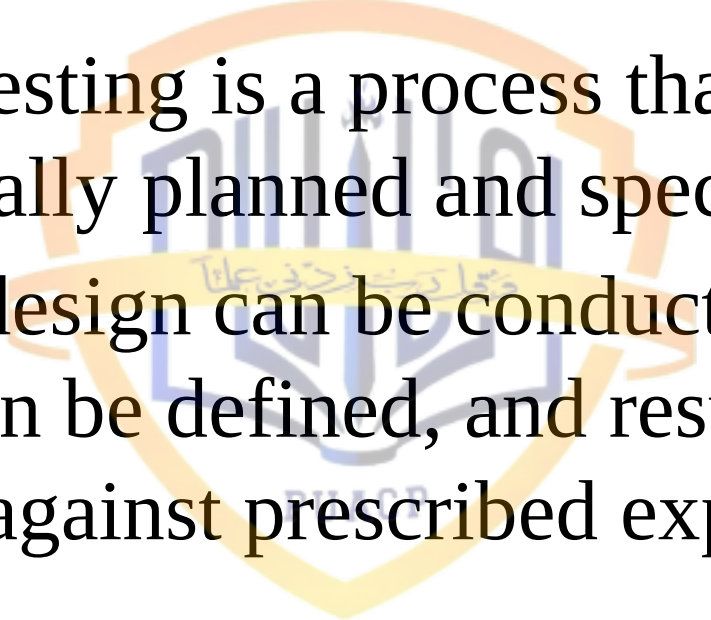
Testing and Litigation of Poor
Software Quality

Testing and Litigation of Poor Software Quality

	Reliable Software	Software Involved in Litigation for Poor Quality
Subroutine testing	Used	Used
Unit testing	Used	Used
New function testing	Used	Rushed or omitted
Regression testing	Used	Rushed or omitted
Integration testing	Used	Used
System testing	Used	Rushed or omitted
Performance testing	Used	Rushed or omitted
Capacity testing	Used	Rushed or omitted

The Art of Debugging



- 
- The logo of the University of Al-Qadisiyah is a circular emblem. It features a central shield with Arabic calligraphy. The shield is surrounded by a ring containing the university's name in Arabic. The entire logo is rendered in a light, semi-transparent blue and yellow color scheme.
- Software testing is a process that can be systematically planned and specified
 - Test case design can be conducted, a strategy can be defined, and results can be evaluated against prescribed expectations

Debugging - 1

- Debugging occurs as a consequence of successful testing
- That is, when a test case uncovers an error
- Debugging is the process that results in the removal of the error
- Although debugging can and should be a process, it is still very much an art

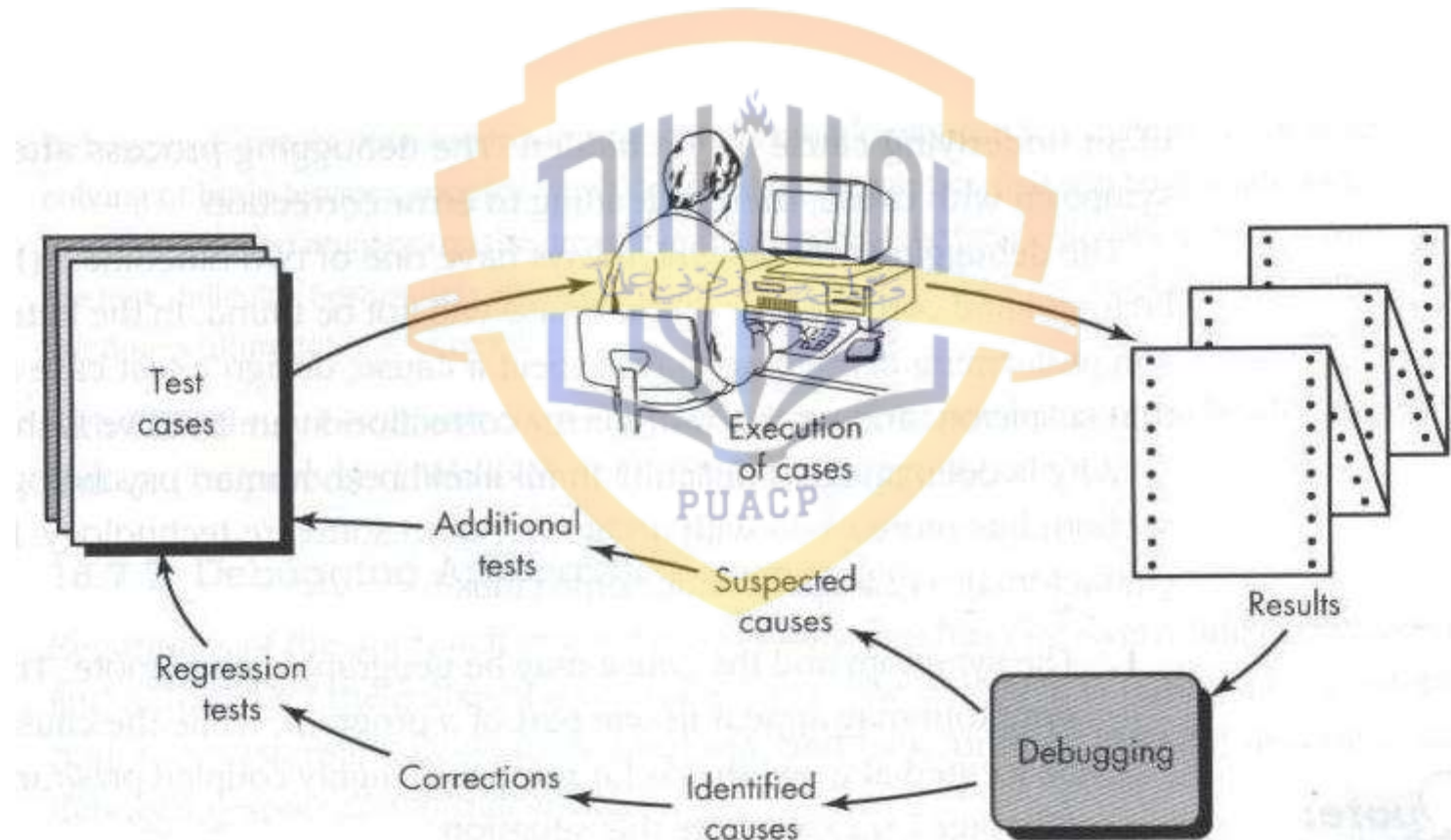
Debugging - 2

- A software engineer, evaluating the results of a test, is often confronted with a “symptomatic” indication of a software problem
- That is, the external manifestation of the error and the internal cause of the error may have no obvious relationship to one another

Debugging - 3

- The poorly understood mental process that connects a symptom to a cause is debugging
- Debugging is not testing but always occurs as a consequence of testing

The Debugging Process



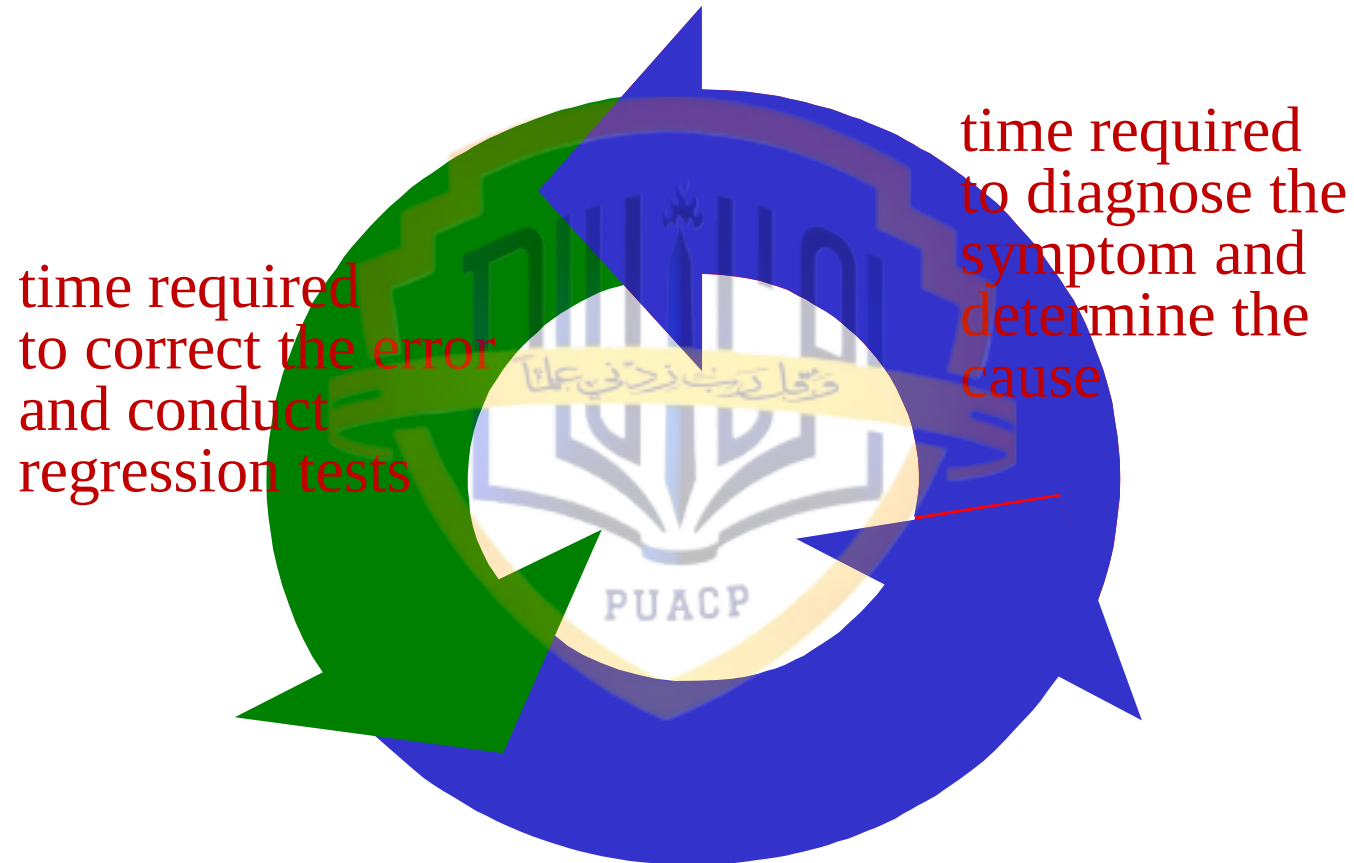
The Debugging Process - 1

- Begins with the execution of a test case
- Results are assessed and a lack of correspondence between expected and actual performance is encountered
- In many cases, the non-correspondence data are a symptom of an underlying cause as yet hidden
- The debugging process attempts to match symptom with cause, thereby leading to error correction

The Debugging Process - 2

- The debugging process will always have one of two outcomes
 - The cause will be found and corrected
 - The cause will not be found
- In the second case, the person performing debugging may suspect a cause, design a test case to help validate that suspicion, and work toward error correction in an iterative fashion

Debugging Effort



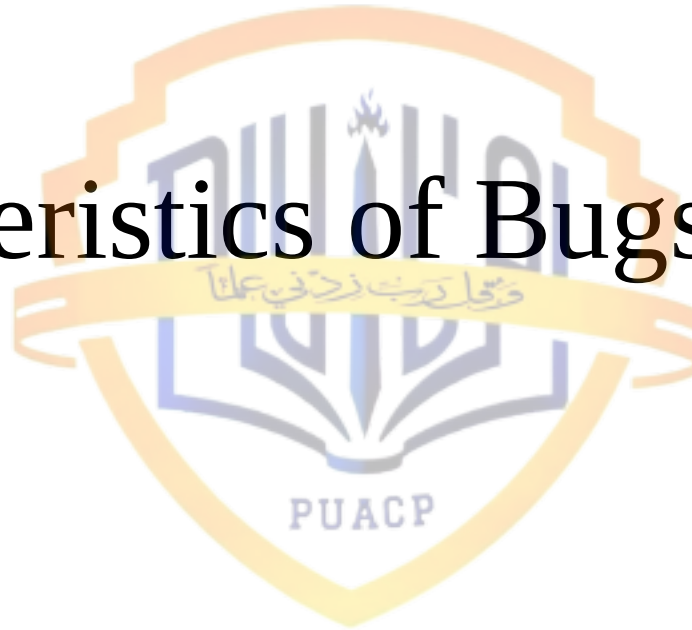
The Debugging Process - 3

- The debugging process is a human-oriented process
- Some people are good at it and others aren't
- Large variances are reported in debugging ability for programmers with the same education and experience

Debugging

- Debugging is one the more frustrating parts of programming. It has elements of problem solving or brain teasers, coupled with the annoying recognition that you have made a mistake. Heightened anxiety and unwillingness to accept the possibility of errors increases the task difficulty. Fortunately, there is a great sigh of relief and a lessening of tension when the bug is ultimately corrected

Characteristics of Bugs/Defects



Characteristics of Bugs - 1

- (1) The symptom and cause may be geographically remote. That is, the symptom may appear in one part of the program, while the cause may actually be located at a site that is far removed. Highly coupled program structures exacerbate this situation

Characteristics of Bugs - 2

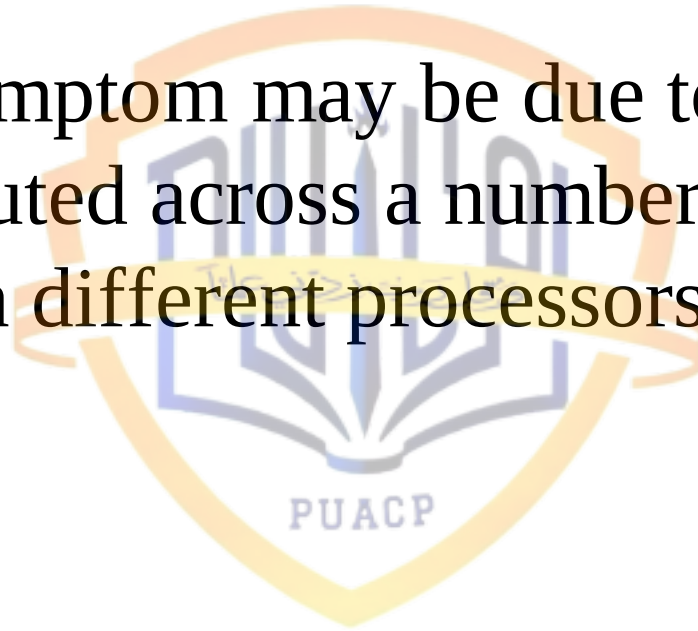
- (2) The symptom may disappear (temporarily) when another error is created
- (3) The symptom may actually be caused by non-errors (e.g., round-off inaccuracies)
- (4) The symptom may be caused by human error that is not easily traced
- (5) The symptom may be a result of timing problems, rather than processing problems

Characteristics of Bugs - 3

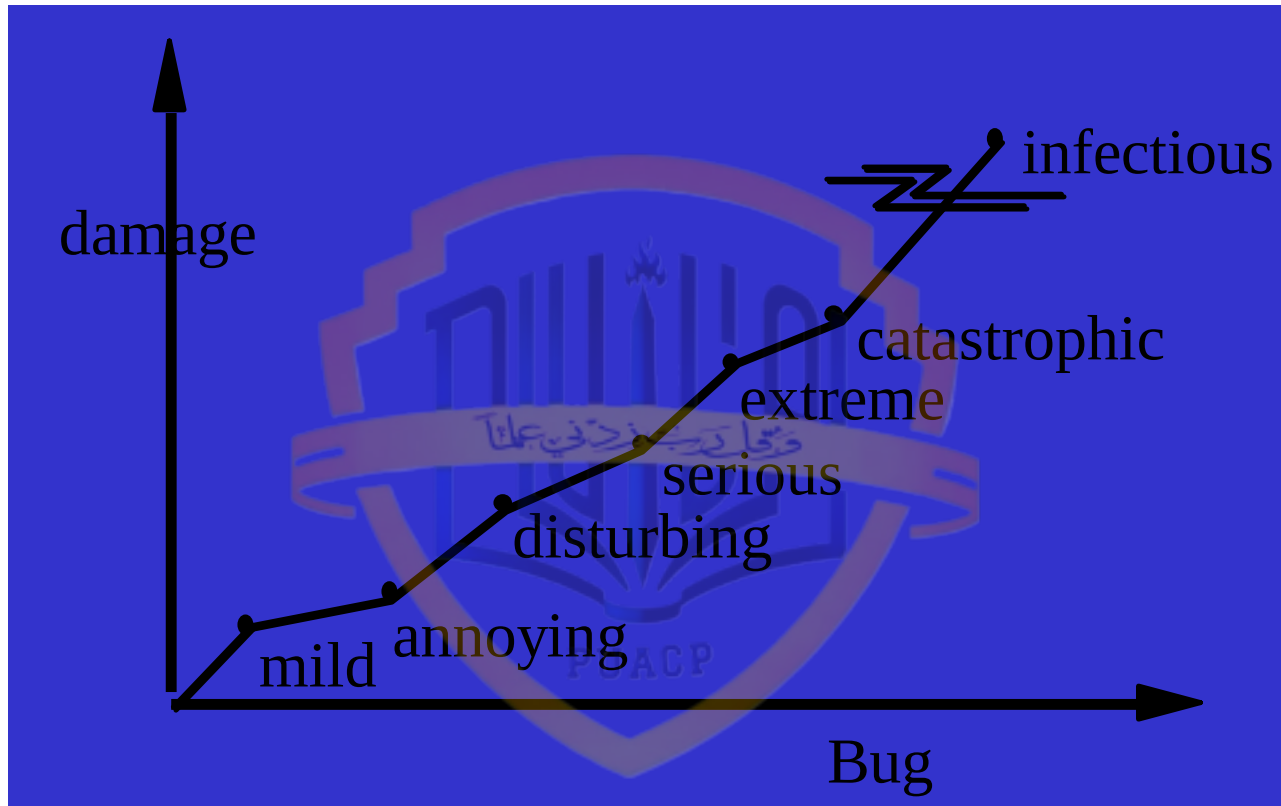
- (6) It may be difficult to accurately reproduce input conditions (e.g., a real-time application in which input ordering is indeterminate)
- (7) The symptom may be intermittent. This is particularly common in embedded systems that couple hardware and software inextricably

Characteristics of Bugs - 3

- (8) The symptom may be due to causes that are distributed across a number of tasks running on different processors



Consequences of Bugs



Bug function-related bugs,
Categories: system-related bugs, data bugs, coding bugs,
design bugs, documentation bugs, standards
violations, etc.

Debugging Approaches - 1

- Regardless of the approach that is taken, debugging has one overriding objective: to find and correct the cause of a software error
- The objective is realized by a combination of systematic evaluation, intuition, and luck

Debugging Approaches

- Brute force
- Backtracking
- Cause elimination



Brute Force Debugging - 1

- The brute force debugging is probably the most common and least efficient method for isolating the cause of a software error
- We apply brute force debugging methods when all else fails, using a “let the computer find the errors” philosophy, memory dumps are taken, run-time traces are invoked, and the program is loaded with WRITE statements

Brute Force Debugging - 2

- We hope that somewhere in the morass of information that is produced we will find a clue that can lead us to the cause of an error
- Although the mass of information produced may ultimately lead to success, it more frequently leads to wasted effort and time
- **Thought** must be expended first

Backtracking Debugging

- Backtracking is a fairly common debugging approach that can be used successfully in small programs
- Beginning at the site where a symptom has been uncovered, the source code is traced backward (manually) until the site of the cause is found
- As the number of source lines increases, the number of potential backward paths may become unmanageably large

Cause Elimination Debugging - 1

- Cause elimination is manifested by induction or deduction and introduces the concept of binary partitioning
- Data related to the error occurrence are organized to isolate potential causes
- A “cause hypothesis” is devised and the aforementioned data are used to prove or disprove the hypothesis

Cause Elimination Debugging - 2

- Alternatively, a list of all possible causes is developed and tests are conducted to eliminate each
- If initial tests indicate that a particular cause hypothesis shows promise, data are refined in an attempt to isolate the bug

Fixing a Bug

- Once a bug has been found, it must be corrected
- We know that at this time, new defects can be introduced
- So, we need to ask three questions before we fix a bug or repair a software defect

Questions before Correction

- Is the cause of the bug reproduced in another part of the program?
- What “next bug” might be introduced by the fix I’m about to make
- What could we have done to prevent this bug in the first place?

Summary

- Today we have discussed debugging – the process of removing defects from software after software testing has been successfully exercised and some observations of failures have been made
- Debugging is a human-oriented activity and productivity varies substantially, even for people with similar education and training

References

- Software Engineering: A Practitioner's Approach by Roger S. Pressman (Chapter 18.5 – 18.7)

